# MZ2POL-09: Migrating Management Zone-Scoped Alerting and Notifications

> **Series:** MZ2POL — Management Zone Migration: Segments, Policies, Workflows | **Notebook:** 10 of 17 | **Created:** July 2026 | **Last Updated:** 10/23/2026

## Overview

Management Zones do three jobs. Notebooks 02–04 and 06–08 cover the **access-control** job; notebook 05 covers the **filtering** job. This notebook covers the third: **alerting**.

An alerting profile scoped to a Management Zone decides who gets paged. When the zone goes away, that scoping has to land somewhere — and it does not land on Segments. This notebook covers where it actually lands, what it costs, and the one step of the migration whose failure mode is undocumented.

> ⚠️ **If you read one thing here, read §2.** The claim that alerting profiles will be "bound to Segments" circulated widely — including in earlier revisions of this series — and it is wrong. Teams that believe it *defer* the alerting work, waiting for a feature that is not coming. That deferral is what turns a manageable migration into a cutover crisis, because the alerting work is the long pole, not the short one.

---

## Table of Contents


1. The Third Job
2. What Actually Replaces an Alerting Profile
3. Inventory Before You Start

4. Converting MZ Rules into Routing Dimensions
5. Building the Replacement Workflow
6. Capability Regressions
7. Problem Visibility Is a Separate Axis
8. Cutover Sequencing and the Deletion Test
9. Validating the Migration
10. Checklist

---

## Prerequisites

| Requirement | Details |
|---|---|
| **Environment** | Dynatrace Gen3 SaaS with Grail |
| **Permissions** | Workflow creation; Settings read for alerting-profile export; IAM policy authoring for §7 |
| **Non-prod tenant** | **Required** for §8 — the deletion test must not be run first in production |
| **Prior reading** | **MZ2POL-01** §5 (the MZ-job-to-successor mapping) · **MZ2POL-03** (assessment) · **MZ2POL-05** (the filtering job, for contrast) |
| **Companion series** | **WFLOW-02** (trigger configuration) · **WFLOW-04** (routing patterns) · **ALERT-01** (alerting architecture) · **ALERT-03** (destinations and cost) |
| **Tagging prerequisite** | The routing dimensions must reach the **problem record** — as primary Grail fields or `primary_tags.*` set on the data, not auto-tags — before any workflow can filter on them. See §4 and MZ2POL-10 |

## 1. The Third Job

A Management Zone conflates three concerns that are three separate mechanisms on Gen3.

| Axis | Question it answers | Gen3 mechanism | Where it is covered |
|---|---|---|---|
| **Filtering** | What does a user *see* in the UI? | Segments | MZ2POL-05 |
| **Access** | What data may a user *read*? | Policies + boundaries on `dt.security_context` | MZ2POL-02/04 |
| **Routing** | Who gets *paged*? | **Problem-triggered workflows** | **This notebook** |
| **Visibility** | Who may *see the problem* at all? | IAM record-level policy on `dt.security_context` | §7 below |

Routing and visibility are related but distinct, and §7 explains why conflating them produces a model that silently does not work.

## Why the alerting job is the hard one

Filtering and access both migrate to constructs that can evaluate a **condition**. A segment can filter on a field condition such as `primary_tags.team = "payments"` or `dt.host_group.id = "PROD-*"`; a boundary can test a `dt.security_context` prefix.

A workflow trigger cannot. It evaluates the **problem record** — an affected-entity tag filter plus an optional DQL matcher, compiled into one expression *"limited to 1,000 characters across all trigger fields."* It can only test fields the problem carries, so a Management Zone whose rules are *computed* — name patterns, entity selectors — has nothing for a trigger to filter on until the value is set on the data as a primary field or tag.

That asymmetry is the single biggest source of surprise in this part of the migration, and §4 is devoted to it.

> **Sources:** Segment limits (DT docs) — *"Only operators* `=` *, and* `in()` *are supported."*; *"You can use wildcards* `*` *for* `starts-with` *,* `contains` *, and* `ends-with` *matches."* Event trigger (DT docs) — *"limited to 1,000 characters across all trigger fields."*

# 2. What Actually Replaces an Alerting Profile

## It is not a segment

There is **no segment field on an alerting profile, and no segment field on a workflow trigger.** Segments are query-time filter conditions — they scope what a *query* returns and what a *user sees*. They do not scope what *fires*.

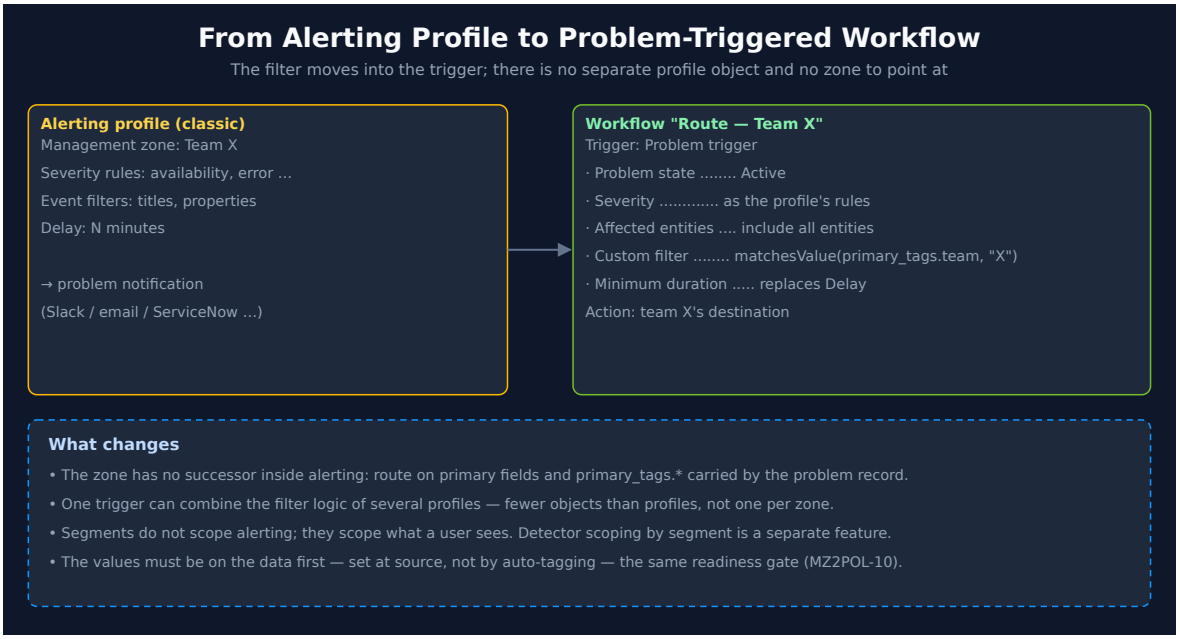
Dynatrace's alert-notification upgrade guide moves that filtering into the workflow trigger itself: *"A workflow's Problem trigger filters problems directly with DQL matchers on the problem."* There is no management-zone filter to carry over, because *"There is no separate filter object to create, name, and maintain, nor is there a one-management-zone-per-profile constraint."*

Segments touch alerting in exactly one documented place: **scoping an anomaly detector** (Anomaly Detection app → *Set scope* → *Segments*). That scopes **detection** — which signals are evaluated — not **routing**.

> **Where anomaly detectors are created (SaaS 1.344).** *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings. Because Anomaly Detection is deprecated, we highly recommend that you use Settings to access your existing configurations and create new ones."* SaaS 1.344 rolls out to tenants in stages — check your tenant's version before following either path. On earlier versions the **Anomaly Detection** app is where custom alerts are created, and the *Set scope* path above is in that app.
>
> **Sources:** Anomaly Detection (DT docs) — *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings."*
>
> **Where the misconception came from.** The claim traces to a Dynatrace community forum answer rather than to documentation, and it propagated into this series: MZ2POL-01, -03, -05, -06, and -99 all carried some version of "alerting profiles → Segments (upcoming)" until July 2026. Independent corroboration that it was never the direction: Dynatrace's current alerting and notifications hub links to problem triggers, Davis event triggers, the Anomaly Detection app, and workflow connectors — and does not link to alerting profiles at all.

## It is a problem-triggered workflow

An alerting profile scoped to MZ *X*, routing to team *X*'s channel, becomes:




No separate profile object exists in this shape. **The filter *is* the trigger.** Dynatrace notes that *"a single workflow trigger can encapsulate the combined filter logic of multiple alerting profiles."*

## Status of the classic surfaces

Neither alerting profiles nor classic problem notifications are deprecated. Both are labeled **Dynatrace Classic**, and the docs recommend simple workflows for new setups. No end-of-life date has been published for either.

The forcing function is not deprecation — it is **the Management Zone itself**. A profile scoped by an MZ is only as durable as that MZ. Profiles scoped purely by severity rules and tags are unaffected by this migration and can be left alone.

> **Sources:** Upgrade guide — alerting and notifications (DT docs) — where filtering goes in a workflow, and that alerting profiles and problem notifications *"continue to work and are not being removed on a published schedule, but they'll not receive new capabilities."* Event trigger (DT docs) — trigger filter surface and DQL-matcher scope. Alerting profiles (DT docs) — Dynatrace Classic status. Use segments in anomaly detection (DT docs) — the detector-scoping exception.

# 3. Inventory Before You Start

Alerting-profile configuration is not queryable via DQL — it lives in Settings. Export it via the Settings API ( `builtin:alerting.profile` ) or the UI, the same way MZ2POL-00 exports Management Zones.

Collect these five columns for every profile. Each one drives a decision later, and several surface gaps that are much cheaper to find now than at cutover.

| Column | Why it matters | Drives |
|---|---|---|
| **Management zone reference** | Identifies which profiles this migration actually touches. Profiles without one need no work | Scope |
| **Severity rules** (level + tag filters) | Carries forward directly onto the workflow trigger | §5 |
| `delayInMinutes` per severity rule | Maps onto the trigger's **Minimum duration** option, which accepts fixed values only (§6.1). Every non-zero value needs a recorded disposition | §6 |
| **Event filters** (predefined / custom title / description) | Becomes the DQL matcher on the trigger | §5 |
| **Notification destinations** consuming this profile | Three destination types (Trello, VictorOps, xMatters) have no native connector; Opsgenie moves to the Jira Service Management connector | §6 |

## The three questions the inventory must answer

1. **How many profiles reference a Management Zone?** This is the real work list — often far smaller than the total profile count.
2. **How many use a non-zero `delayInMinutes`?** Every one needs a Minimum duration value chosen deliberately — the allowed values are fixed, so some delays will not map exactly (§6.1).
3. **What destinations are in play?** Trello, VictorOps, and xMatters need HTTP rebuilds; Opsgenie moves to the Jira Service Management connector (§6.2).

> **Consolidate while you inventory.** Profiles are frequently near-duplicates that differ only by zone. The target is one workflow per **team/channel**, not one per profile —

> see §5. A hundred MZ-scoped profiles routinely collapse to a much smaller set of destinations.

# 4. Converting MZ Rules into Routing Dimensions

This is the actual work, and it is enrichment work, not alerting work.

## Route on what the problem record carries

A problem trigger evaluates the **problem record**. Dynatrace's tagging guidance draws the line between the two models in one row: classic setups *"Route alerts based on tags applied to entities"*; on Latest Dynatrace you *"Route alerts on the primary fields and tags carried by the alert events themselves."* The alert-notification upgrade guide confirms what reaches the problem: *"Primary Grail fields and tags (* `primary_tags.*` *),* `dt.security_context` *, and the Cost Allocation fields are automatically propagated from the alerting events into the problem record."* It also lists the classic `entity_tags` field as *"still exists but is deprecated"*.

Auto-tags are not a route. *"Auto-tagging rules have no effect on Smartscape on Grail and are not used in any Latest Dynatrace app."* A dimension that exists only as an auto-tag today has to be set on the data — at the source or centrally — before a workflow can route on it.

> **What the docs do not say.** The event-trigger page describes the *Affected entities* tag filter's three modes but not which tag source it evaluates — classic entity tags, `entity_tags` on the problem, or `primary_tags.*`. Until that is documented, put routing conditions in the **Additional custom filter query**, where the field is explicit, and confirm any tag filter against real problems with *Query past events* before relying on it.

## The conversion table

Classify every MZ rule by whether its dimension already reaches the problem record. The right-hand column is your prerequisite backlog.

| MZ rule shape | On the problem record today? | What to do |
|---|---|---|
| Host group membership | ✅ Primary field `dt.host_group.id` | Route on it directly |
| Kubernetes namespace / cluster | ✅ Primary fields `k8s.namespace.name`, `k8s.cluster.name` | Route on them directly |
| `dt.security_context` already set at the source | ✅ Yes | Usable directly; also the visibility field (§7) |
| Host or process tag `team:checkout` | ⚠️ Only as a classic tag | Set it as `primary_tags.team` at the source or through central enrichment (MZ2POL-10 §5) |
| Service name **contains** `payment` | ❌ Computed | Set a value at the source, or with a central / OpenPipeline enrichment rule (MZ2POL-10 §5, MZ2POL-11) |
| Entity selector expression | ❌ Computed | Same — translate the rule to a value on the data (MZ2POL-11 §7) |
| Process/host **name pattern** | ❌ Computed | Same |

Every ⚠️ and ❌ row is work that must complete — **and reach new problems** — before the corresponding workflow can be built. Enrichment is forward-only: problems opened before the value existed never carry it.

## Auditing routing coverage on problem records

Measure on the problems themselves, not on entities. Entity-side tags tell you nothing about what a trigger can see: on the validation tenant on 10/13/2026, classic tags carried a `Team:` value on 28 of 35 services, while `smartscapeNodes "SERVICE"` returned `tags[team]` on **0 of 35** — and neither number says what reached a problem.

Replace `team` / `Team:` with whichever key carries your routing dimension.

> **Sources:** Differences between classic auto-tagging and primary Grail tags (DT docs) — *"Route alerts on the primary fields and tags carried by the alert events themselves."*; *"Auto-tagging rules have no effect on Smartscape on Grail and are not used in any Latest Dynatrace app."* Upgrade guide — alerting and notifications (DT docs) — *"Primary Grail fields and tags (`primary_tags.*`), `dt.security_context`, and the Cost Allocation fields are automatically propagated from the alerting events into the problem record."* Event trigger (DT docs) — the *Affected entities* tag-filter modes.

```
// Routing-tag coverage on PROBLEM RECORDS: the classic tag vs the primary tag
that replaces it.
// Executed 10/13/2026 over 7 days: 5,293 problems / classic Team: tag on 514
(9.7%) / primary_tags.team on 0.
// entity_tags is the deprecated classic field; primary_tags.&lt;key&gt; is
what a Latest Dynatrace route reads.
fetch dt.davis.problems, from:-7d
| fieldsAdd classic_team = iAny(startsWith(entity_tags[], "Team:")),
            primary_team = isNotNull(primary_tags.team)
| summarize {problems = count(),
             classic_team_tag = countIf(classic_team == true),
             primary_team_tag = countIf(primary_team)}
| fieldsAdd classic_pct = round(classic_team_tag * 100.0 / problems, decimals:
1),
            primary_pct = round(primary_team_tag * 100.0 / problems, decimals:
1)
```

`primary_pct` is the number that matters. Below 100, the gap is your §4 backlog: problems without the value match no workflow that routes on it and page nobody. On the validation tenant the classic tag reached 9.7% of problems and the primary tag none — a tenant that would route nothing if it cut over on `primary_tags.team` today.

The built-in primary fields need no enrichment, so audit them the same way — they are often the fastest route to coverage:

```
// Primary-field coverage on problem records. On dt.davis.problems these
fields are ARRAYS
// (values accumulate across the events grouped into the problem) — hence
arraySize().
// Executed 10/13/2026 over 7 days: 5,293 problems / namespace 31.3% / host
group 2.4% / security context 26.3%.
fetch dt.davis.problems, from:-7d
| summarize {problems = count(),
             with_namespace = countIf(arraySize(k8s.namespace.name) &gt; 0),
             with_host_group = countIf(arraySize(dt.host_group.id) &gt; 0),
             with_context = countIf(arraySize(dt.security_context) &gt; 0)}
| fieldsAdd namespace_pct = round(with_namespace * 100.0 / problems, decimals:
1),
            host_group_pct = round(with_host_group * 100.0 / problems,
decimals: 1),
            context_pct = round(with_context * 100.0 / problems, decimals: 1)
```

> ⚠️ **On a problem record these fields are arrays, and `==` silently matches nothing.** The upgrade guide: *"Values accumulate across every event grouped under the problem."* On the validation tenant over 7 days, `k8s.namespace.name == "otel-demo"` matched **0** problems while `matchesValue(k8s.namespace.name, "otel-demo")` and `in("otel-demo", k8s.namespace.name)` each matched **1,202** — with no notification on the `==` form (10/13/2026). Write the trigger's custom filter as `matchesValue(primary_tags.team, "checkout")`, and confirm it with *Query past events* before saving.
>
> **Sources:** Upgrade guide — alerting and notifications (DT docs) — *"Values accumulate across every event grouped under the problem."* Counts executed against the validation tenant, 10/13/2026.

## New values reach new problems only

Setting a value at the source or in a central enrichment rule does not rewrite problems that already exist. Re-run the coverage queries over a window that starts after the change, and wait until `primary_pct` reaches 100 before building the workflow that depends on it — a trigger filtering on a value that only some problems carry will silently under-route.

# 5. Building the Replacement Workflow

## The full trigger filter surface

These are all the controls a problem trigger offers:

| Setting | What it does |
|---|---|
| **Problem state** | *Active*, *Active or closed*, or *Closed* |
| **Categories** | Davis problem categories |
| **Severity** | Minimum severity threshold |
| **Affected entities** | Filter by entity tags — three modes: include all / all defined tags / any defined tag. The page does not say which tag source it evaluates (§4) |
| **Additional custom filter query** | A DQL **matcher** on the problem record — a *subset* of DQL: no aggregation, no querying across a set of events |
| **Minimum duration** | Postpones the trigger until the problem has been open for a fixed duration (§6.1) |
| **Updates** | Re-trigger when selected fields change; the *affected entities* field tracks the union of `affected_entity_ids` and `smartscape.affected_entities` |
| **Wait for root cause analysis** | The trigger *"starts only after Dynatrace Intelligence has completed root cause analysis for the problem"* — the page recommends enabling it |

No management-zone filter. No segment filter. No entity-selector filter.

All of these compile into one matching expression, *"limited to 1,000 characters across all trigger fields"*; a trigger that exceeds it fails validation on save. A long list of tag values or entity conditions is the usual way to hit it — route on one primary tag instead (§4).

> **SaaS 1.348 (rollout start 10/12/2026, staged tenant rollout):** *"Davis events and problems no longer default* `event.severity` *to* `3`*."* A severity filter carried over from a profile that never set severity explicitly may stop matching once 1.348 reaches your tenant. The release note's action plan: *"If it was relying on the default, change the filter to Any severity so the workflow continues to fire after the upgrade."* For each

> converted trigger, decide whether its severity filter is intentional. Until 1.348 arrives, the current defaulting still applies — verify your tenant's version.
>
> **Sources:** Event trigger (DT docs) — the problem-state options; *"When enabled, the trigger starts only after Dynatrace Intelligence has completed root cause analysis for the problem."*; *"limited to 1,000 characters across all trigger fields."* SaaS 1.348 (DT docs) — *"Davis events and problems no longer default* `event.severity` *to* `3` *."*

## Granularity: one simple workflow per destination

| Approach | Object count | Verdict |
|---|---|---|
| One workflow per Management Zone | = MZ count | ❌ Unmanageable; most would be near-duplicates |
| **One simple workflow per team/channel** | = destination count | ✅ Recommended |
| One workflow branching to all teams | 1 | ❌ Single point of failure; multi-step billing; hard to review |

A **simple** workflow — one trigger, one notification action — avoids the workflow-hour consumption that multi-step workflows incur. Adding an HTTP action or branching logic can move it into the billed category. ALERT-03 covers that boundary; re-verify the current billing rule against your contract before committing to a fleet size.

## Do not route on the management-zone payload field

`event()["management_zones"]` looks like a natural routing key, but on a Gen3 problem record it is not there to read. On the validation tenant, **0 of 15,875** problems over 30 days carried a `management_zones` field (10/05/2026) while management zones still existed, and the semantic dictionary has no such field. An expression built on it has nothing to match today — before any zone is deleted — so a workflow that depends on it either never notifies or fails at evaluation. Route on primary fields and `primary_tags.*` in the DQL matcher instead (§4–§5).

> **Dictionary:** no row for `management_zone` / `managementzone` under `filter contains(name, …)`, read 10/05/2026 (control: `dt.security_context` ( `stable` ) returned by

> the same query). Field-presence counts from `fetch dt.davis.problems, from:-30d` with `affected_entity_ids` as the positive control (15,875 of 15,875).

Silent alert loss is the worst failure mode available in this migration, and this is the easiest way to cause it. WFLOW-04 §4 documents the pattern as legacy for exactly this reason.

### Manage as code

At any meaningful profile count, hand-building in the UI produces drift within weeks. Template the first workflow, then generate the rest — see **AUTOM-04** (Terraform) or **AUTOM-03** (Monaco).

## 6. Capability Regressions

One thing gets worse: three destinations lose their native connector. Duration-based suppression — listed here as a regression until 09/2026 — carries over, with one wrinkle. Both are cheaper to plan for than to discover.

### 6.1 Duration-based suppression maps onto Minimum duration

An alerting profile can delay notification until a problem has been open longer than *N* minutes ( `delayInMinutes` ). Teams use it to suppress transient blips.

**The workflow model does have an equivalent — the trigger's Minimum duration option.** The problem trigger's **Minimum duration** option (renamed from **Delay** in 08/2026) postpones *"the trigger until the problem has been open for at least the configured duration"* — 5, 10, 15, 30, 60, 120, 240, 1440, or 10080 minutes, evaluated on `dt.duration_marker` , and *"the trigger fires once when the threshold is crossed on the active phase"* — and where the filter also detects the closed phase, *"the trigger will additionally fire once when the problem is closed"*

> **The documentation now agrees (09/17/2026).** Until 09/07/2026 the alert-notification upgrade guide said the classic **Duration** filter had no alternative, and this section treated duration suppression as the migration's one hard regression. The guide was rewritten. It now lists `dt.duration_marker` as *"How long the problem has been open, as a stepped threshold"*, says *"The delay, update, and severity capabilities described in this guide exist only on the workflow trigger."*, and names

> *"separate profiles created purely to vary the delay"* among the Classic constraints that no longer exist.

**The wrinkle: the values are fixed.** Minimum duration takes only the nine values above. A profile whose delay is not one of them — 7 or 20 minutes, say — has no exact match, and rounding changes behavior — up suppresses a little more, down pages a little sooner. Decide each one deliberately rather than letting a script round it.

Per-profile options:

| Option | When it fits | Trade-off |
|---|---|---|
| **Map to the nearest Minimum duration** | The delay is load-bearing and close to an allowed value | Rounding up suppresses slightly more; rounding down pages slightly sooner |
| **Drop the delay** | Delay was short; the signal is genuinely actionable | Slight increase in short-lived pages |
| **Replace with a Davis anomaly detector** | The signal was noisy because a static threshold was wrong for it | Changes detection semantics, not just routing; needs tuning |
| **Express as an SLO burn-rate alert** | The concern was sustained degradation, not a threshold crossing | Best conceptual fit; requires an SLO to exist (**SLO-04**) |
| **Retire the alert** | The delay was masking an alert nobody acts on | Free noise reduction — check usage first |

**Delay-dependent profiles need no special wave.** Decide each one from the table above and convert it alongside everything else. Profiles with `delayInMinutes: 0` — usually the large majority — need no decision at all.

> **A long `delayInMinutes` is usually evidence the alert was the wrong *shape*, not merely delayed.** A profile suppressing 30 minutes of a firing condition is describing a burn-rate concern. Route those to SLO burn-rate alerts where an SLO exists, and to Davis where one does not.

## 6.2 Three destinations have no native connector

Documented connector mapping:

| Classic notification | Workflow equivalent |
|---|---|
| Ansible | RedHat Ansible connector |
| Custom integration | HTTP Request action |
| Email | Email or Microsoft 365 connector |
| Jira | Jira connector |
| PagerDuty | PagerDuty connector |
| ServiceNow | ServiceNow connector |
| Slack | Slack connector |
| Microsoft Teams | Microsoft Teams connector |
| Opsgenie | Jira Service Management connector (Opsgenie's successor; SaaS 1.343) |

**Trello, VictorOps, and xMatters have no dedicated connector** — the guide: *"No dedicated connector. Use HTTP Request."*

**Opsgenie moves to the Jira Service Management connector.** Atlassian is retiring Opsgenie in favour of Jira Service Management (JSM), and Dynatrace ships a Jira Service Management Connector (DT docs): *"The Jira Service Management Connector integrates Workflows with Jira Service Management (JSM) to automate the creation, enrichment, acknowledgment, and closing of alerts."* It arrived with SaaS 1.343 (DT docs) (staged rollout) — confirm it is in your tenant. The alert-notification upgrade guide (DT docs) now agrees: *"Opsgenie is being retired by Atlassian and replaced by Jira Service Management (JSM). Use Jira Service Management Connector or HTTP Request."*

Each of the three becomes an HTTP-action rebuild: reconstruct the payload against the destination's current API, store credentials in the vault, and accept that the workflow may now count as multi-step for billing.

Rebuild the payload against the destination's current API contract — do not port the old webhook body verbatim.

### Awareness-only recipients and personal notifications

Some people were added to an MZ-scoped email profile purely for awareness, not as responders. The Problems app has offered them a personal email subscription to a filter, with no workflow and no configuration permission — *"The email notification is directly triggered within OpenPipeline, meaning only simple filters can be applied."*

> **Problems app personal notifications — deprecated from SaaS 1.347.** SaaS 1.347 (staged tenant rollout from 09/16/2026) deprecates them: *"Personal problem notifications in Problems, are deprecated. You can no longer create new ones, and existing notifications continue to work but prompt you to turn them off. The recommended alternative is Workflows."* Verify your tenant's version. Before 1.347 reaches your tenant, a personal subscription still works for an awareness-only recipient; after it, that recipient needs a simple email workflow of their own. Either way, do not plan an MZ migration around **new** personal subscriptions. (The Problems app page still describes the feature without a deprecation note — the release note is the newer source.)
>
> **Sources:** Problems app (DT docs) — *"The email notification is directly triggered within OpenPipeline, meaning only simple filters can be applied."* SaaS 1.347 (DT docs) — *"For each personal problem notification you rely on, build a simple workflow in Workflows that sends you email alerts for problems, then turn off the corresponding personal problem notification in Problems."*

## 7. Problem Visibility Is a Separate Axis

If a Management Zone was doing double duty — restricting which problems a team could **see**, not just which ones paged them — then replacing the routing does **not** replace the visibility.

| | **Routing** | **Visibility** |
|---|---|---|
| Mechanism | Workflow trigger filter | IAM record-level policy |
| Field | Primary fields / `primary_tags.*` in the DQL matcher | `dt.security_context` |
| Failure if wrong | Wrong team paged | Wrong team **sees data they should not** |




## The constraint that decides the design

When multiple events aggregate into a problem, the values of a given field are combined into an **array**. Because of how Grail record-level permission filters currently work, **only `dt.security_context` supports filtering array values.**

> A problem-visibility model built on team tags, a custom attribute, or any field other than `dt.security_context` **will not work**. This is not a performance note or a preference — the filter will not behave correctly once events aggregate.

Dynatrace propagates `dt.security_context` from an event's `dt.source_entity` onto alerting events specifically so IAM policy rules can control alert and problem visibility at the record level (SaaS 1.333).

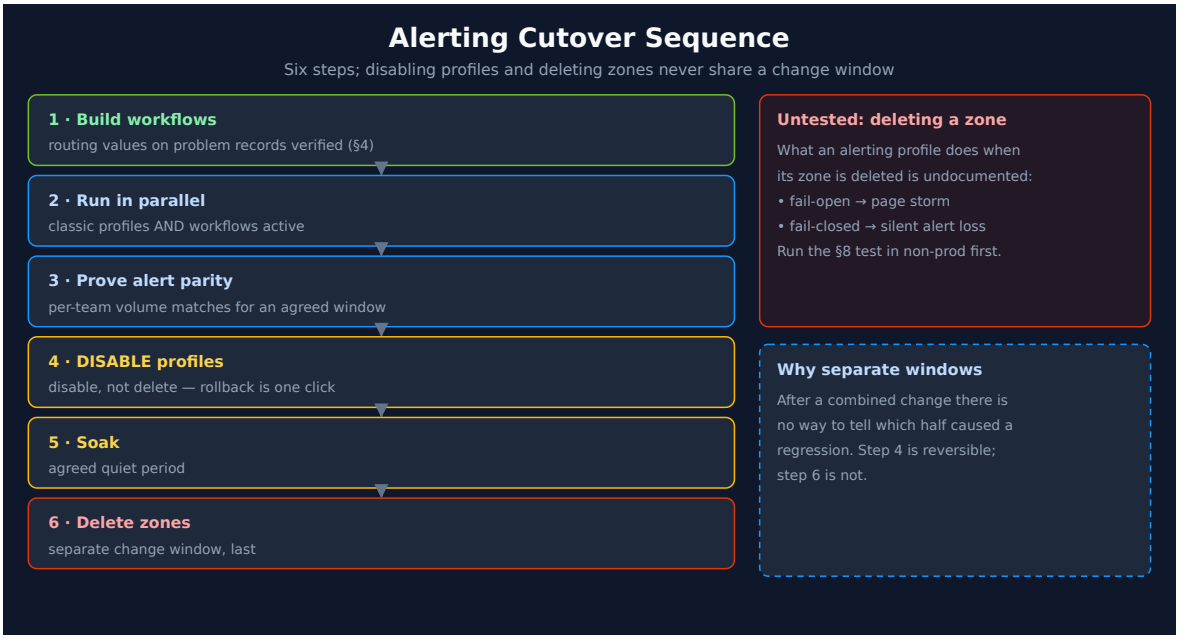
Problem records hold `dt.security_context` as an array (every problem record that carried it on the validation tenant, 10/05/2026), so the operator matters as much as the field: Grail's permission docs state *"Using `=`, `STARTSWITH` or `IN` when the field holds an array will always return `false`. If you expect your record filters might contain an array, use the `MATCH` operator in your IAM statements."* A visibility policy written as `storage:dt.security_context = "team-a"` hides the team's own problems.

This is the same field the access-control half of the migration uses, which is convenient: MZ2POL-02 and MZ2POL-04 already establish it. If those notebooks' work is done, visibility is largely already handled — verify rather than rebuild.

> **Sources:** Problems app (DT docs) — the array-filtering constraint. SaaS 1.333 release notes (DT docs) — `dt.security_context` propagation onto alerting events. Permissions in Grail (DT docs) — *"Using `=`, `STARTSWITH` or `IN` when the field holds an array will always return `false`."*

# 8. Cutover Sequencing and the Deletion Test

## The order




Steps 4 and 6 must not share a change window. After a combined change there is no way to tell which half caused a regression.

## The unresolved question

> **⚠️ Undocumented behavior — test it, do not assume it.**
>
> **What happens to an alerting profile whose Management Zone reference is deleted?**
>
> - **Fails open** — the profile matches everything. Across a large profile estate, a page storm.
> - **Fails closed** — the profile matches nothing. Silent alert loss, which is worse, because nothing announces it.

> This is not documented, and it could not be verified while writing this notebook. Establish it yourself before deleting any production Management Zone.

## The test procedure

Run in a **non-prod tenant**.

| Step | Action | Record |
|---|---|---|
| 1 | Create a throwaway MZ scoped to a small, controllable entity set | MZ name, scope |
| 2 | Create an alerting profile scoped to that MZ, routing to a test destination | Profile config |
| 3 | Trigger a problem on an **in-scope** entity; confirm the notification fires | Positive baseline |
| 4 | Trigger a problem on an **out-of-scope** entity; confirm it does **not** fire | Negative baseline |
| 5 | Delete the Management Zone. Change nothing else | Deletion timestamp |
| 6 | Re-run step 3 | Fires / does not fire |
| 7 | Re-run step 4 | **This is the answer.** Fires ⇒ fail-open. Does not fire ⇒ fail-closed |

Step 4 is the one people skip, and it is the one that matters — without a negative baseline, step 7 proves nothing.

## What the answer changes

| Result | Consequence |
|---|---|
| **Fail-open** | MZs must be deleted only *after* profiles are deleted, not merely disabled — otherwise every dangling profile pages everyone |
| **Fail-closed** | Safer, but demands parity proof *before* deletion, since failures will be silent |

### Retire Management Zones last

Classic apps still depend on Management Zones, and so do any alerting profiles you chose to keep. MZ2POL-05 §8 covers coexistence; the ordering rule is the same here — the MZ is the last thing to go.

## 9. Validating the Migration

### Alert parity is the acceptance test

The only meaningful proof is that each team receives the same pages after cutover as before. Compare per-team alert volume across the parallel-run window.

> **This is the one thing the platform team cannot validate alone.** Only the team currently receiving an MZ-scoped page can confirm they still receive the equivalent page. Budget for their time explicitly — this is the most common source of cutover slippage.

### Problem distribution by security context

Establishes the baseline distribution to compare against after cutover:

```
// Problem-event volume by security context over the parallel-run window.
// Executed 07/31/2026 over 7 days; results below the cell.
// Counts problem-EVENT records, not distinct problems - see the note that
follows.
fetch events, from: -7d
| filter event.kind == "DAVIS_PROBLEM"
| expand dt.security_context
| summarize problems = count(), by:{dt.security_context}
| sort problems desc
| limit 20
```

`dt.security_context` is an **array** on problem records once events aggregate — hence the `expand`. This is the same array behavior that constrains the visibility model in §7.

## Security-context coverage on problems

A problem with no security context cannot be filtered for visibility and may not route as expected:

Executed against a validation tenant over 7 days on 07/31/2026, the top rows were:

| `dt.security_context` | problem-event records |
|---|---|
| *(null)* | 2,319,333 |
| `ENV \| Prod` | 147,055 |
| `Elevate \| Prod \| Monitoring Team` | 77,689 |
| `Elevate \| Prod` | 73,533 |
| `Prod All (old)` | 73,522 |

**Read the unit before you read the numbers.** `fetch events | filter event.kind == "DAVIS_PROBLEM"` returns the raw problem-event stream — many records per problem, one for each state update over its life — so these are *records*, not distinct problems. On the same tenant and window, `fetch dt.davis.problems` returned 3,318 rows against 2,480,695 problem-event records: an over-count of roughly 750×. That is fine for a *relative* comparison across contexts, which is what this baseline is for, and badly wrong if you quote it as a problem count to anyone. ALERT-99 §3 sets out all three Davis surfaces and which question each answers.

Note also the shape of that top row: the largest bucket by far is the one with **no security context at all**.

```
// Security-context coverage across the problem-event stream.
// dt.security_context is an ARRAY here: count non-empty arrays, not non-null values.
// Executed 10/13/2026 over 7 days: total 66,803 / with_context 15,813 / 23.7%.
fetch events, from: -7d
| filter event.kind == "DAVIS_PROBLEM"
| summarize {total = count(), with_context = countIf(arraySize(dt.security_context) &gt; 0)}
| fieldsAdd coverage_pct = round(with_context * 100.0 / total, decimals: 1)
```

On the validation tenant, 10/13/2026, over 7 days:

| total | with_context | coverage_pct |
|---|---|---|
| 66,803 | 15,813 | **23.7** |

**76.3% of the problem-event stream carries no security context at all.** (The 07/31/2026 run of the earlier form of this query read 6.5%; the tenant's event volume and enrichment have both changed since, so compare runs on your own tenant, not against these figures.) That number is the argument of this notebook in a single figure.

The migration story people reach for is that MZ-scoped alerting will be rebuilt on a field like `dt.security_context` — swap one scoping dimension for another and the routing follows. This measurement is what that plan collides with. A field populated on under a quarter of records is not a scoping dimension yet; it is an enrichment project that has not happened. Build routing on it today and three quarters of the stream falls through to whatever the default is — which, for alerting, means either everyone is paged or nobody is.

That is why §4 puts enrichment first and treats it as the prerequisite rather than a parallel workstream. **Run this query before you commit to a cutover date, not after.** A tenant at a quarter coverage has substantial enrichment work in front of it; a tenant already near 100% can move quickly. The number is cheap to obtain and it is the difference between a schedule and a guess.

A coverage figure this low is also worth a second look before you treat it as pure bad news: check whether the contexts that *are* populated cover the entities that actually page people. Coverage weighted by alert-carrying entities is the number that matters operationally, and it is often better than the raw record-level figure — the null bucket is inflated by high-frequency events on unenriched infrastructure.

## What to check before declaring done

| Check | Pass condition |
|---|---|
| Routing coverage on problem records (§4) | `primary_pct` at 100 for every routing key, measured over a window that starts after enrichment |
| Workflow per destination exists | Every inventoried destination has a live workflow |
| Alert parity per team | Within agreed tolerance across the parallel window, **confirmed by the receiving team** |
| No workflow reads `management_zones` | Zero matches across all workflow definitions |

| Check | Pass condition |
| --- | --- |
| `delayInMinutes` dispositions | Every non-zero value has a recorded Minimum duration decision |
| Security-context coverage | 100%, or every gap explained |
| Deletion behavior established | §8 test run and result recorded |

## 10. Checklist

**Inventory**

- [ ] Export alerting profiles ( `builtin:alerting.profile` ) — Settings API or UI
- [ ] Identify which profiles reference a Management Zone (the real work list)
- [ ] Record `delayInMinutes` per severity rule
- [ ] Record every notification destination and its type
- [ ] Identify near-duplicate profiles that collapse to one destination

**Enrichment (prerequisite)**

- [ ] Classify every MZ rule by whether its dimension reaches the problem record (§4)
- [ ] Set every missing dimension on the data — at the source or with central enrichment (MZ2POL-10 §5), not with auto-tags
- [ ] Run the problem-record coverage queries until they reach 100%
- [ ] Measure over a window that starts after enrichment — existing problems are not rewritten

**Build**

- [ ] One simple workflow per team/channel — not per Management Zone
- [ ] Trigger routes with a custom filter query on primary fields or `primary_tags.*` — `matchesValue()` , never `==` (problem fields are arrays)
- [ ] *Wait for root cause analysis* decided per trigger (the docs recommend enabling it); each trigger under the 1,000-character limit
- [ ] Carry severity rules across from the old profile
- [ ] Rebuild the three destinations with no dedicated connector (Trello, VictorOps, xMatters) as HTTP Request actions; move Opsgenie to the Jira Service Management connector

- [ ] Template and manage as code (AUTOM-03 / AUTOM-04)
- [ ] Verify no workflow reads `event()["management_zones"]`

**Regressions**

- [ ] Every non-zero `delayInMinutes` has a recorded disposition
- [ ] On-call teams told about any delay that was rounded *before* cutover
- [ ] Awareness-only recipients: a simple email workflow each (SaaS 1.347 deprecates Problems-app personal notifications — §6)

**Visibility**

- [ ] Visibility model uses `dt.security_context` and nothing else
- [ ] IAM record-level policies in place and tested

**Cutover**

- [ ] Deletion-behavior test run in non-prod; result recorded (§8)
- [ ] Parallel run complete; alert parity confirmed **by the receiving teams**
- [ ] Classic profiles **disabled**, not deleted
- [ ] Soak period elapsed
- [ ] Management Zones deleted in a **separate** change window, last

---

# Summary

1. **Alerting is the third job a Management Zone does**, and it migrates to problem-triggered workflows — not to Segments. Segments scope queries and anomaly detectors; they never scope triggers, notifications, or visibility.
2. **The real work is enrichment.** Triggers evaluate the problem record; MZ rules are computed conditions on entities. Every routing dimension must be set on the data as a primary field or `primary_tags.*` — auto-tags do not reach Grail — and must reach new problems before its workflow can exist.
3. **One confirmed capability regression:** three notification destinations have no native connector (Opsgenie moves to the JSM connector). Duration-based suppression maps onto the trigger's **Minimum duration** option, and since its 09/07/2026 rewrite the upgrade guide agrees.
4. **Visibility is a separate axis** and works only on `dt.security_context`, because it is the only field that filters correctly once events aggregate into a problem.

5. **The deletion failure mode is undocumented.** Test it in non-prod before touching production, and never delete zones and profiles in the same change window.

## Next Steps

- **MZ2POL-16** — the other zone-scoped alerting artifacts: metric events (the transform drops the zone), maintenance windows, classic SLOs, notification payloads, and proving parity
- **MZ2POL-06** — migration execution, phasing, and rollback for the migration as a whole
- **MZ2POL-07** — validation and troubleshooting
- **MZ2POL-99** — the consolidated best-practice checklist across the series

## Additional Resources

- Upgrade guide — alerting and notifications (DT docs) — the authoritative old→new mapping: where filtering goes, the connector table, and delay as a workflow-trigger capability
- Event trigger (DT docs) — problem-trigger filter surface and DQL-matcher scope
- Alerting and notifications (DT docs) — the current Gen3 hub
- Alerting profiles (DT docs) — Dynatrace Classic status
- Problem notifications (DT docs) — classic notification status
- Problems app (DT docs) — array-filtering constraint; personal email subscription (deprecated from SaaS 1.347)
- SaaS 1.347 release notes (DT docs) — Problems-app personal notifications deprecated; migrate to Workflows
- SaaS 1.348 release notes (DT docs) — `event.severity` no longer defaulted to `3`
- Use segments in anomaly detection (DT docs) — the one documented segment/alerting intersection
- SaaS 1.333 release notes (DT docs) — `dt.security_context` propagation onto alerting events

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace*

*documentation.*